

Recurrent Neural Networks - BPTT (Backpropagation Through Time)

- PRASANNA KOIRALA • ROLL 20016
- AIAC 536 NATURAL LANGUAGE PROCESSING
- KATHMANDU UNIVERSITY

* The Setup

An RNN processes a three-step sequence (x_1, x_2, x_3) . The hidden state updates as $h_t = f(x_t, h_{t-1})$ and one scalar loss is taken at the end: $L = L(h_3)$. Throughout I use the standard concrete form:

$$h_t = \tanh(w_{hh} \cdot h_{t-1} + w_{xh} x_t + b)$$

→ Why RNNs are needed. A bag of words or an average of word vectors discards word order entirely. "good, not bad" and "bad, not good" produce the identical vector. An RNN instead reads the sequence one step at a time and carries a memory forward, so order is preserved by construction.

* Q1 Forward Pass

Start from $h_0 = 0$ (no context yet), then repeat one identical operation per step:



$$h_1 = \tanh(w_{hh} h_0 + w_{xh} x_1 + b)$$

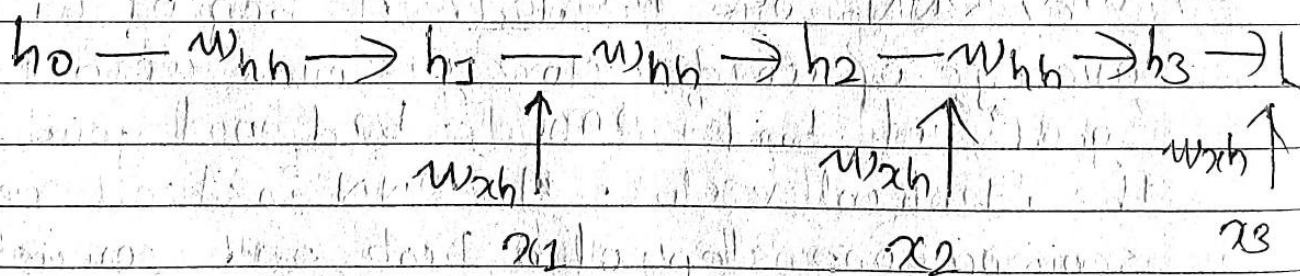
$$h_2 = \tanh(w_{hh} h_1 + w_{xh} x_2 + b)$$

$$h_3 = \tanh(w_{hh} h_2 + w_{xh} x_3 + b)$$

$$l = L(h_3)$$

→ Three things follow directly:

- Each step needs the previous one finished — h_2 cannot be computed before h_1 exists. The forward pass cannot be parallelized across time.
- The same w_{hh} , w_{xh} , b appear in every line — one parameter set, reused at every step (Q6).
- The loss is taken only at the end, yet h_1 and h_2 must still be corrected. That is the problem BPTT solves (Q3).



Boxes for $h_0 \dots h_3$ and $x_1 \dots x_3$, horizontal arrows labelled w_{hh} , vertical arrows labelled w_{xh} , one arrow from h_3 to l . Used a ruler — this is the diagram the question explicitly asked for.

Reading it: two arrows enter every hidden

state — one from that step's input, one from the previous hidden state. Only h_3 connects to the loss. The horizontal chain is the only route by which x_1 can influence L , and read backwards, the only route by which L can reach x_1 's parameters.

* Q2 Hidden State Dynamics

h_t is the network's memory — a fixed-size summary of everything seen up to time t . Its size never changes, whether the network has read 3 words or 300. Constant memory cost, but finite capacity.

How context accumulates. Substituting each equation into the next:

$$h_2 = \tanh \left(\underbrace{w_{hh} \cdot [\tanh(w_{hh} h_0 + w_{xh} x_1 + b)]}_{\text{this is } h_1} + w_{xh} x_2 + b \right)$$

~~~~~  $\Rightarrow$  this highlighted is  $h_1$

Expanded fully,  $h_3$  is a nested function of  $x_1$ ,  $x_2$  and  $x_3$ . Every input still has influence, but older information has been squeezed through  $\tanh$  one more time at each step. That is the intuition behind vanishing gradients in Q7 — the maths there states the same more fully.

### \* Q3 Backpropagation Through Time (BPTT)

The loss is measured only at  $h_3$ , but the weights that produced  $h_1$  and  $h_2$  are equally responsible. So the error signal must travel backwards, along the same chain the forward pass came along:

$$L \rightarrow h_3 \rightarrow h_2 \rightarrow h_1$$

Why "Through Time". In a feed-forward network, backpropagation travels backwards through layers — structures existing simultaneously in space. An RNN has only one cell, what looks like a stack of layers is the same cell at different moments in time. Travelling backwards therefore means stepping backwards through timestamps, not layers.

Hence backpropagation through time. (BPTT)

Mechanically, it is still just the chain rule. Two consequences follow from the structure, and each gets its own question: every hidden state must be stored (Q5), and the shared weights accumulate a gradient at every step. (Q6)

# \* Q4 Chain Rule Derivation

$L$  depends on  $h_1$  only through  $h_2$ , which affects  $h_3$ . Applying the multivariable chain rule along that single path:

$$\partial L / \partial h_1 = (\partial L / \partial h_3) \cdot (\partial h_3 / \partial h_2) \cdot (\partial h_2 / \partial h_1)$$

Step 1 - the loss term

with  $L = \|h_3\|^2 = \sum_i h_{3,i}^2$ :

$$\frac{\partial L}{\partial h_3} = 2h_3$$

Step 2 - one step of recurrence (the important one)

Write the pre-activation  $a_t = w_{hh} h_{t-1} + w_{xh} x_t + b$ , so  $h_t = \tanh(a_t)$ .

Then 
$$\frac{\partial h_t}{\partial h_{t-1}} = \text{diag}(\tanh'(a_t)) \cdot w_{hh}$$

Using the identity  $\tanh'(a) = 1 - \tanh^2(a)$   

$$= 1 - h_t^2$$

$$\frac{\partial h_t}{\partial h_{t-1}} = \text{diag}(1 - h_t^2) \cdot w_{hh}$$

← the Jacobian of one timestep

Step 3 - assemble

$$\frac{\partial L}{\partial h_3} = \underbrace{\frac{\partial L}{\partial h_3}}_{\frac{\partial L}{\partial h_3}} \cdot \underbrace{\text{diag}(1 - h_3^2) w_{hh}}_{\frac{\partial h_3}{\partial h_2}} \cdot \underbrace{\text{diag}(1 - h_2^2) w_{hh}}_{\frac{\partial h_2}{\partial h_1}}$$

General form, for a sequence of length  $T$

$$\frac{\partial L}{\partial h_k} = \left( \frac{\partial L}{\partial h_T} \right) \cdot \prod_{t=k+1}^T \text{diag}(1 - h_t^2) w_{hh}$$

THIS PRODUCT IS THE CENTER OF THE WHOLE ASSIGNMENT

Verified: we computed  $\partial L / \partial h_3$  twice - once with PyTorch's autograd, once by evaluating the formula above directly. Maximum difference:  $1.79 \times 10^{-7}$ , i.e. floating-point noise.

The derivation is correct, not merely plausible.

## \* Q5. Intermediate Activations & Memory

Look again at the Jacobian from Q4:

$$\frac{\partial h_t}{\partial h_{t-1}} = \text{diag}(J \cdot h_t^2) \text{ when } h_t = \tanh(h_{t-1})$$

It contains  $h_t$  itself. The backward pass cannot evaluate this term without the numerical value the forward pass produced. So every hidden state must be kept, not discarded after use - the forward pass is not only computing an answer, it is recording the working the backward pass will need.

The cost! memory  $\sim T \times H$ , i.e. linear in sequence length. A 1,000 step sequence stores 1,000 hidden states before a single gradient is computed. This is real engineering constraint, and it is why truncated BPTT (backpropagate only the last  $K$  steps) and gradient checkpointing (store some states, recompute the rest) exist.

## \* Q6 Parameter Sharing

Why the same weights at every step:

- Sequences have no fixed length. Separate weights per timestamp would mean a different model for every input length, and no way to handle anything longer than seen in training.



- A pattern should be recognisable wherever it occurs. "not" should be understood the same way at position 2 or position 40.
- Far fewer parameters. One matrix instead of  $T$ , cheaper and less prone to overfitting.

The effect on gradients — they SUM. Because  $w_{hh}$  is used at every step it has many paths to the loss, and the chain rule requires adding each contribution:

$$\frac{\partial L}{\partial w_{hh}} = \sum_{t=1}^T \left( \frac{\partial L}{\partial h_t} \right) \cdot \left( \frac{\partial h_t}{\partial w_{hh}} \right)$$

Note the summation. A non-shared parameter receives one gradient term; a shared one accumulates  $T$  of them. Two consequences: one update per parameter, not per timestep (all contributions are summed first, then a single update applied); and gradient magnitude scales with sequence length, which is part of why long sequences need careful learning rates and gradient clipping.

# \* Q7 Vanishing & Exploding Gradients

Straight from the general result in Q4, the gradient reaching an early timestep is a product of  $T-k$  matrices. Repeated multiplication is inherently unstable. Letting  $\lambda$  be roughly the largest singular value of one Jacobian:

$$\left\| \frac{\partial l}{\partial h_k} \right\| \sim \lambda^{T-k}$$

| CASE                | WHAT HAPPENS           | CONSEQUENCES                                                   |
|---------------------|------------------------|----------------------------------------------------------------|
| $\lambda < 1$       | vanishes exponentially | early steps get no learning signal - no long range dependences |
| $\lambda \approx 1$ | stable                 | a knife-edge, not a region                                     |
| $\lambda > 1$       | explodes exponentially | enormous updates destroy the weights                           |

why tanh makes vanishing more likely.  $\tanh'(a)$

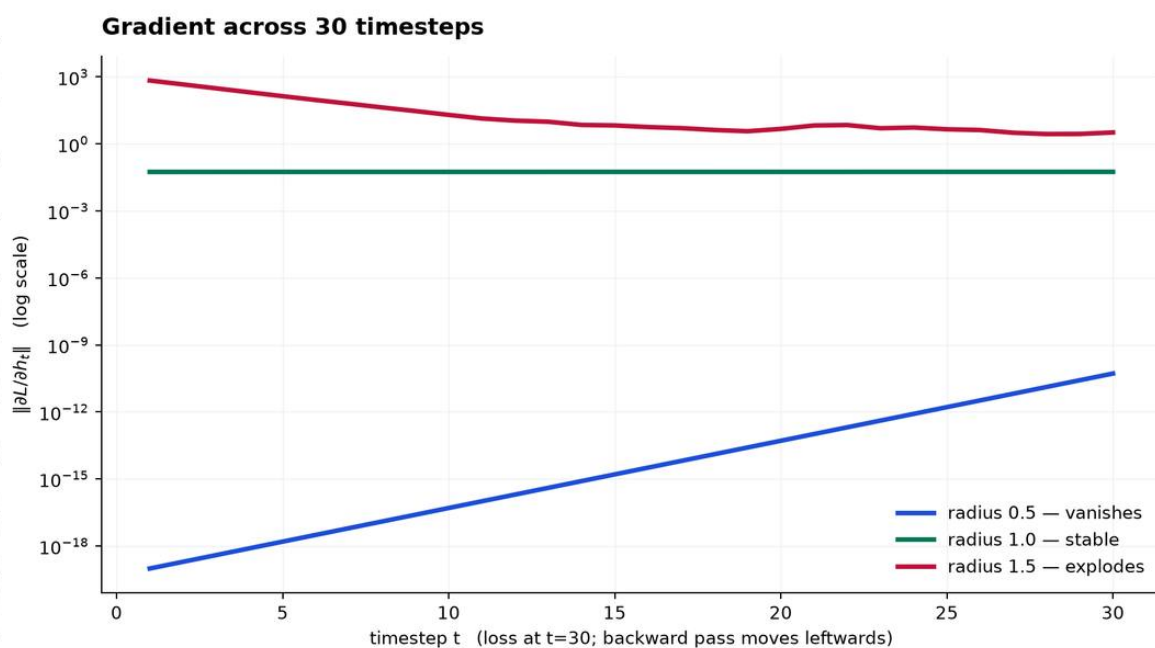
$\tanh'(a) = 1 - \tanh^2(a) \leq 1$ , with equality only at  $a=0$ . So the diag  $(1 - h^2)$  factor is always  $\leq 1$  and usually well below it, pushing

$\lambda$  downwards at every step regardless of  $W_{hh}$ .

→ Measured, not just argued

we ran 30 timesteps with the spectral radius of  $W_{hh}$  fixed exactly, and tracked the gradient norm at every step:

| RADIUS | PREDICTED RADIUS      | MEASURED              | VERDICT                 |
|--------|-----------------------|-----------------------|-------------------------|
| 0.5    | $1.86 \times 10^{-9}$ | $1.86 \times 10^{-9}$ | exact match             |
| 1.5    | $1.28 \times 10^5$    | $2.12 \times 10^2$    | $\sim 600\times$ weaker |



⇒ The KEY FINDING - THE TWO FAILURES ARE NOT SYMMETRIC

Vanishing matches theory exactly; by  $t=1$  the gradient is around  $10^{-19}$ , numerically indistinguishable from no signal. Exploding is roughly 600x weaker than predicted, because as the gradient grows so does  $h_t$ , and as  $h_t$  approaches  $\pm 1$  the factor  $(1 - h_t^2)$  collapses towards zero. The saturating non-linearity acts as its own brake on explosion.

This matters practically: explosion is loud and fixable by gradient clipping, whereas vanishing is silent - no error, no warning, the network simply never learns long-range structure. That is the harder failure, and it is what LSTM and GRU were designed to fix, using additive gated paths so gradients can cross many steps without being multiplied down at each one.

## Q8 PyTorch Implementation

We implemented a custom RNN cell in PyTorch rather than calling `nn.RNN`, so that all three parameters are explicit and each gradient can be inspected separately. The recurrence is written out by hand:

```
class SimpleRNNCell(nn.Module):  
    def __init__(self, input_size, hidden_size):  
        self.w_xh = nn.Parameter(torch.randn(  
            hidden_size, input_size) * 0.5)  
        self.w_hh = nn.Parameter(  
            torch.randn(hidden_size, input_size) *  
                0.5)  
        self.b = nn.Parameter(torch.zeros(hidden_size))  
  
    def forward(self, x, h_prev):  
        return torch.tanh(self.w_xh @ x +  
            self.w_hh @ h_prev + self.b)
```

Run over  $T=5$  timesteps (satisfying  $T \geq 5$ ), a scalar loss  $L = \|h^5\|^2$  is taken at the final step, and `backward()` called. Every parameter received a gradient:

T = 5 timesteps      scalar loss L = ||h\_5||^2 = 1.292772

W\_xh.grad      shape (3, 2)  
[ 0.85000    0.30433]  
[ -0.53082   -0.09728]  
[ -0.39766   -0.15938]  
norm = 1.13573

W\_hh.grad      shape (3, 3)  
[ 0.45752   -0.50366   -0.38502]  
[ -0.66460   0.72272   0.42795]  
[ -0.46181   0.51451   0.32838]  
norm = 1.53108

b.grad      shape (3,)  
[ 0.96634   -0.70100   -0.64621]  
norm = 1.35750

Every parameter received a gradient -> BPTT reached all of them.

⇒ what these gradients physically represent

Each number answers one precise question: if I increased this single parameter by a tiny amount, how much would the loss change?

Formally  $p.\text{grad}[i,j] = \frac{\partial L}{\partial p_{ij}}$

- Sign - positive means increasing the weight increases the loss, so training should decrease it. This is why gradient

descent subtracts:  $p \leftarrow p - \eta \times p \cdot \text{grad}$

- **Magnitude** - how strongly that weight influences the loss; so training should decrease it. Near zero means the loss is flat in that direction.

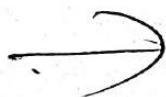
The specifically recurrent part. Because  $w_{hh}$  is shared across all  $T$  steps,  $w_{hh} \cdot \text{grad}$  is not the influence of one use - it is the sum of its influence at every timestep, exactly the summation derived in Q6. A single call to `backward()` silently walked the chain from  $h^5$  back to  $h_1$ , computed a Jacobian at each step, and accumulated  $T$  contributions into one matrix. That is BPTT performed automatically.

## \* Conclusion

Everything in this assignment rests on one object - the product of Jacobians derived in Q4:

$$\frac{\partial L}{\partial h_k} = \frac{\partial L}{\partial h_T} \cdot \sum_{t=k+1}^T \text{diag}(J - h_t^2) w_{hh}$$

Q5 follows because each Jacobian needs its own stored  $h_t$ . Q6 follows  $w_{hh}$  appears in



every factor, so its gradient's sum  $\nabla f$  follows because a product of  $T$  terms changes exponentially in  $T$ . The derivation was verified against autograd to  $10^{-7}$ , and the vanishing/exploding behavior measured rather than assumed — revealing that the two failure modes are not symmetric, since tanh saturation damps explosion but nothing damps vanishing.

Full implementation, runnable notebook and all figures:

[github.com/prasanna-kolra/58-NLP-MTech](https://github.com/prasanna-kolra/58-NLP-MTech)

↓  
assignments / a4 - rnn - bptt /